

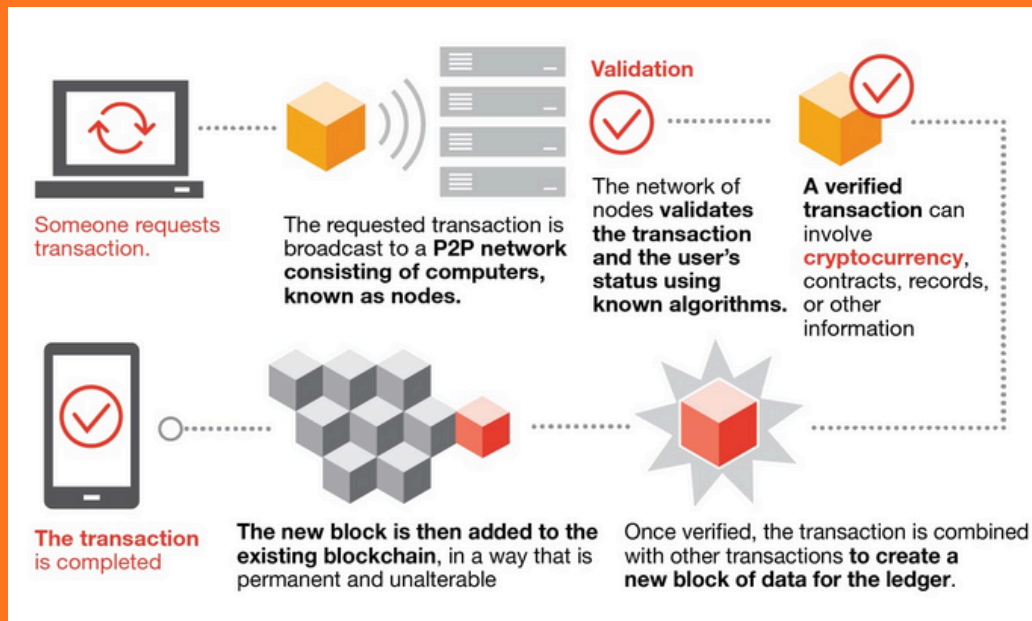
The Double-Spending Problem. Could It Still Threaten Cryptocurrencies?

1

YES

2

How Transactions work?



3

Let's start with the basics of how a blockchain transaction works. Think of a blockchain like a shared, tamper-resistant spreadsheet that thousands of computers keep in sync.

First, someone requests a transaction — for example, I want to send you some digital money. That's the laptop icon on the left. My wallet creates a message that says who I'm paying, how much, and it signs that message with my private key — like sealing a letter with a unique wax stamp.

Next, that transaction is broadcast to a peer-to-peer network of computers, called nodes. No central bank or single server — it's many independent machines talking to each other. Every nearby node hears about my transaction within seconds.

Now comes validation. The nodes run a checklist using well-known rules. They verify the digital signature to make sure the request really came from me. They check that I actually have the funds to spend and that I'm not trying anything sneaky. They also make sure the transaction format follows the protocol's rules. If anything looks wrong, the transaction is ignored. If it looks good, nodes keep passing it along.

Once enough valid transactions pile up, the network groups them together into a block — you can think of a block as one page in that global ledger. Different blockchains use different methods to agree on the next page — you'll hear terms like 'proof of work' or 'proof of stake' — but the goal is the same: the network reaches a

single, shared decision about which block is next.

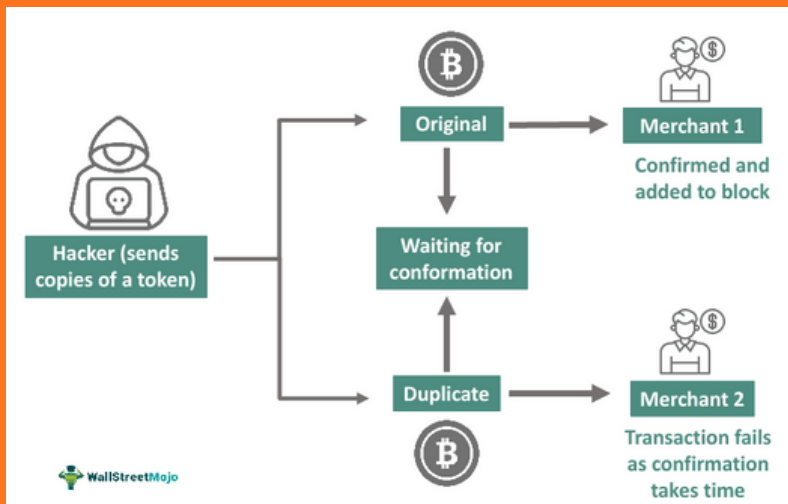
When a block is accepted, it's linked to the one before it, forming a chain of blocks — the blockchain. Each block contains a fingerprint of the previous block. That linking is powerful: change anything on an earlier page and all the fingerprints no longer match. In practice, that makes past records extremely hard to alter.

At that moment, your wallet will show the transaction as 'confirmed.' Over the next few minutes, more blocks are added on top of it, which is like adding extra layers of concrete — the deeper it is in the chain, the harder it is to rewrite. That's why people talk about waiting for a few confirmations.

So, to recap the flow on the slide: a user requests a transaction → it's broadcast to the network → nodes validate it using rules and cryptography → valid transactions are bundled into a block → the network agrees on that block → the block is added to the chain → and the transaction is complete and permanently recorded.

This process isn't just about moving value; it's also what stops 'double spending.' Because everyone agrees on one global order of transactions, once my payment is in the chain, trying to spend the same coins again will be rejected as invalid. We'll dig into that threat next, but this is the foundation that makes the whole system trustworthy

What Is Double-Spending?



Double-spending is spending the same cryptocurrency or blockchain token more than once.

Double-spending is a significant threat to blockchain networks, potentially allowing a token or cryptocurrency to be used more than once, undermining trust in digital transactions.

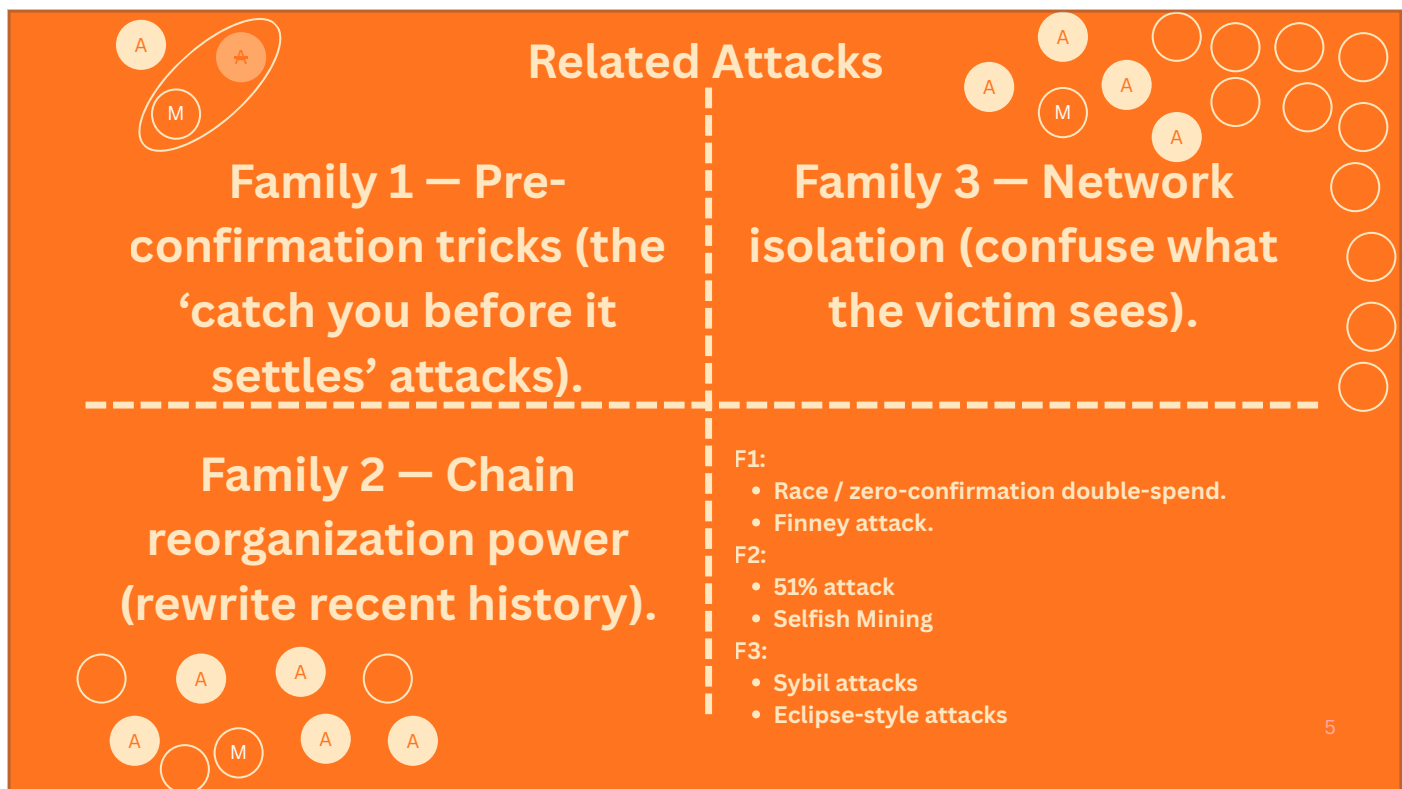
4

Double-spending is simply trying to use the same digital coin twice. With cash, that's impossible—I hand you a \$10 bill and I no longer have it. But a digital file can be copied, so we need rules that stop anyone from 'copy-pasting' value.

Look at the diagram. On the left, the attacker sends two copies of the same coin. Along the top path is the original payment to Merchant 1. Along the bottom path is a duplicate payment to Merchant 2. In the middle you see 'waiting for confirmation'—that short window before the network finalizes a block. That window is where double-spend attempts live.

Here's a simple story. Alice has 1 BTC and buys a laptop from Bob. She broadcasts a transaction sending Bob that 1 BTC. While it's still pending, she also creates a second transaction sending the same 1 BTC to Charlie. Different parts of the network hear about these transactions at slightly different times. Miners or validators will eventually confirm one of them in a block. If Bob's payment confirms first, Charlie's becomes invalid; if Charlie's confirms first, Bob's is rejected. The blockchain records only one outcome.

So why do people worry about double-spending? Because before confirmation, a fast attacker can try to trick a merchant—especially for in-person, instant-delivery sales—by paying with a transaction that hasn't settled yet. If the merchant ships goods on a zero-confirmation payment and the attacker's duplicate is the one that gets confirmed, the merchant loses.



Now that we know what double-spending is, let’s look at the main ways attackers try to make it happen. I’ll group them into three families and call out two enablers.

Family 1 — Pre-confirmation tricks (the ‘catch you before it settles’ attacks).

Race / zero-confirmation double-spend. The attacker broadcasts two conflicting payments at nearly the same time: one to the merchant, one that benefits the attacker—often with a higher fee or marked RBF (replace-by-fee) so miners prefer it. If the merchant hands over goods before a block confirmation, the conflicting tx can win and the merchant is left unpaid.

Defenses: don’t accept high-value payments at zero conf; monitor mempools for conflicts/RBF flags; set risk-based wait times.

Finney attack. A miner pre-mines a block that spends the coins back to themselves, doesn’t publish it yet, pays a merchant with the same coins, gets the goods, then quickly publishes their pre-mined block so the merchant’s payment is invalidated. This only works if the merchant accepts zero-conf.

Defenses: require confirmations (even one confirmation defeats classic Finney).

Family 2 — Chain reorganization power (rewrite recent history).

3) 51% attack. If an attacker controls a majority of block production (hash rate or stake), they can build a private, longer chain, then release it to reorg the public chain. That lets them reverse already confirmed payments and double-spend. This is the

most damaging scenario because it beats confirmations.

Defenses: strong, decentralized consensus; diversified validators/miners; higher confirmation thresholds on smaller chains; reorg monitoring and response policies at exchanges/merchants.

Selfish mining (amplifier). By withholding blocks to gain a lead, a selfish miner increases the rate of short reorgs. Selfish mining doesn't guarantee a double-spend by itself, but it creates conditions—more frequent reorgs—under which a determined attacker can attempt one.

Defenses: protocol tweaks that reduce selfish-mining advantage; healthy network difficulty/participation.

Family 3 — Network isolation (confuse what the victim sees).

5) Sybil/Eclipse-style attacks (enablers). Spinning up many fake nodes or isolating a victim's peers can filter what transactions they see. If a merchant's node is isolated, the attacker can feed it the 'pay-the-merchant' transaction while the broader network mines the conflicting one.

Defenses: diverse peers, peer limits, anti-Sybil controls, use multiple gateways or third-party mempool watchers.

Key takeaway:

Pre-confirmation attacks rely on merchants being in a hurry.

Reorg attacks rely on power—control over block production.

Network attacks rely on visibility—what the victim can or can't see.

Tie this back to prevention: confirmations proportional to value, good mempool monitoring, diversified connectivity, and robust consensus. If a chain is weak or centralized, the window for these attacks widens; if the chain is healthy and the merchant follows sound ops, the window narrows dramatically.”

How To Prevent?

- **Enhanced consensus mechanism**
- **Block timestamps**
- **Transaction copies**
- **Inclusion of nonce**
- **Confirmation of Unspent Transaction Output (UTXO)**

6

Now that we've seen how double-spending works, let's talk about prevention. Think of it as five layers that work together rather than one silver bullet.

First—Enhanced consensus.

Consensus is how the network agrees on a single history. In Proof-of-Work or Proof-of-Stake, independent nodes propose blocks and the network picks one chain as the winner. By requiring economic work or stake and by rewarding honest behavior, consensus makes it very hard for an attacker to rewrite recent blocks. Practically, this is why we 'wait for confirmations'—each block that lands on top of a payment makes a successful rewrite exponentially harder.

Second—Block timestamps.

Timestamps give every block a rough place in time and help nodes detect weird ordering or stale blocks. They aren't a stand-alone anti-fraud tool—miners can't set arbitrary times—but they support ordering, auditability, and fork resolution. In short, timestamps help the network converge quickly, which shrinks the attacker's window.

Third—Transaction copies (network propagation).

Every node keeps a copy of unconfirmed transactions in its mempool and relays them. Because thousands of independent nodes see the same mempool, they can spot conflicts—two transactions trying to spend the same coin—and reject the later one. Good propagation and mempool monitoring reduce the chance a merchant accepts a payment that's about to lose to a conflicting spend. If you're a merchant,

use tools that flag conflicts or Replace-By-Fee (RBF) attempts.

Fourth—Inclusion of a nonce.

‘Nonce’ shows up in two helpful ways. In Proof-of-Work, a block nonce is part of the puzzle that secures consensus—no nonce, no valid block. In account-based systems like Ethereum, a per-account transaction nonce means each transaction must have a unique sequence number. That uniqueness prevents replay—an attacker can’t simply resend an old transaction or create two with the same number.

Fifth—Confirmation of the UTXO (Unspent Transaction Output).

In UTXO systems like Bitcoin, every coin you control is an output with a unique ‘serial number.’ Nodes maintain a UTXO set and will only accept a spend if the referenced output is currently unspent. Once your transaction is confirmed in a block, that output is marked spent and can’t be used again. Waiting for a few confirmations closes the door on attacks like the classic Finney attack, which only works when merchants accept zero-confirmation payments.

So the takeaway is layered defense: strong consensus to make history hard to rewrite, timestamps to aid ordering, broad propagation to surface conflicts, nonces to guarantee uniqueness or secure block creation, and the UTXO model to enforce ‘spent once’ at the protocol level. Add practical policies—like required confirmations proportional to value and mempool-conflict alerts—and you dramatically reduce double-spend risk.

So... Does double-spending still threaten cryptocurrencies?

- **Ethereum Classic (ETC)** — Jan-2019: ~219,500 ETC double-spent (~\$1.1M). Jul–Aug-2020: ~807k ETC and ~458k ETC double-spent across multiple 51% attacks (multi-thousand-block reorgs).
- **Bitcoin Gold (BTG)** — May-2018: ~\$18M double-spent; additional incidents reported in 2020.
- **Horizen/ZenCash** — Jun-2018: ~\$550k double-spent.
- **Monacoin** — May-2018: deep reorg event enabling double-spend attempts.

7

Let me close with the question in our title: Does double-spending still threaten cryptocurrencies?

Short answer: yes, but mostly on smaller or poorly secured chains and in zero-confirmation situations—not on the major networks when you follow best practices.

Evidence it still happens.

Ethereum Classic (ETC) has suffered multiple confirmed 51% attacks with real double spends: ~219,500 ETC (~\$1.1M) in Jan 2019, then ~800,000 ETC (~\$5.8M) on July 31, 2020, and ~460,000 ETC (~\$3.2M) on Aug 5, 2020. These were deep chain reorgs observed by Coinbase.

Medium

+1

Bitcoin Gold was hit in May 2018 with an estimated \$18M double spend and was later delisted by Bittrex over the incident.

Cointelegraph

+1

Other smaller PoW coins (e.g., ZenCash/Horizen, Monacoin) also saw successful double-spend events in 2018 totaling hundreds of thousands of dollars.

Distributed Networks Institute

+1

How big is the risk on major chains?

For Bitcoin-style PoW, the math shows the risk falls off exponentially with each confirmation. If an attacker controls 10% of hashrate, the chance of beating 6 confirmations is about 0.02% (≈ 1 in 5,000). With lower attacker power, it drops even further.

arXiv

Claims of Bitcoin “double-spends” occasionally surface during short reorgs, but investigations (e.g., Jan 2021) showed no successful double-spend—just normal chain reorganization behavior plus replace-by-fee on unconfirmed transactions.

CoinDesk

For Ethereum’s Proof-of-Stake, once a block is finalized, reverting it would require burning at least one-third of all staked ETH—an economically prohibitive attack by design.

Kiln

Operational reality.

Pre-confirmation (zero-conf) payments remain the soft spot across networks; attackers race a conflicting transaction or exploit RBF policies to try to replace an unconfirmed payment. The fix is procedural: wait for confirmations proportional to value and monitor mempools for conflicts.

Bitcoin Optech

+1

Bottom line for the slide:

Yes, double-spending exists, and we still see real losses on smaller PoW chains with low hashpower.

CoinDesk

For top networks (Bitcoin, Ethereum), when you require normal confirmations/finality, the statistical risk is tiny and the economic cost to attack is massive.

arXiv

+1

Therefore: the threat is not gone, but it’s manageable with sound protocol choices

and sane payment policies.”

References

- <https://blog.college.ch/blockchain-technology/all-you-need-to-know-about-cryptocurrency-and-blockchain-technology/>
- <https://www.investopedia.com/terms/d/doublespending.asp>
- <https://www.wallstreetmojo.com/double-spending/>
- <https://en.wikipedia.org/wiki/Double-spending>
- <https://www.geeksforgeeks.org/ethical-hacking/what-is-double-spending-in-blockchain/>
- <https://chatgpt.com/>

Thanks!!!!!!!!!!!!!!!!!!!!!!!!!!!!

!!

!!

!!